

TITAN GO: FIELD OPERATIONS APP UPGRADE PLAN

Serviceman Mobile App → Voice-First, Offline-First Field Operating System

****System:**** Titan Go (Part of TitanBOS - Titan Business Operating System)

****AI Core:**** Titan Zero (Cognitive decision engine)

****Timeline:**** 20 weeks (5 months)

****Team:**** 2-3 developers + 1 designer + 1 QA

****Total LOC:**** 25,000-30,000 lines added

****Platforms:**** iOS + Android (Flutter)

PHASE 0: FOUNDATION (Weeks 1-3)

Security Hardening + Voice Engine Setup

0.1 Critical Security Fixes (6 hours)

...

Week 1:

- |— Move hard-coded API keys to environment variables
 - |— Update android/build.gradle for --dart-define
 - |— Update ios/Runner configs
 - |— Test on physical devices
 - |— Time: 2 hours
- |— Implement token encryption (flutter_secure_storage)
 - |— Replace SharedPreferences for auth tokens
 - |— AES-256 encryption
 - |— Handle existing user migration
 - |— Time: 2 hours
- |— Add certificate pinning
 - |— pin_flutter package
 - |— Pin API server certificate
 - |— Test cert rotation
 - |— Time: 2 hours

...

0.2 Voice Engine Implementation (30 hours)

****Core Voice Architecture:****

```dart

// VoiceEngine - handles all voice I/O

class VoiceEngine extends GetxService {

late SpeechToText speechToText;

late FlutterTts flutterTts;

final isListening = false.obs;

```
final voiceInput = ".obs";
final voiceError = Rxn<String>();
```

```
@override
void onInit() {
 super.onInit();
 _initializeSpeechRecognition();
 _initializeTextToSpeech();
}
```

```
Future<void> _initializeSpeechRecognition() async {
 speechToText = SpeechToText();
```

```
 final available = await speechToText.initialize(
 onError: (error) => voiceError.value = error.errorMsg,
 onStatus: (status) => print('Speech status: $status'),
);
```

```
 if (!available) {
 voiceError.value = 'Speech recognition not available';
 }
}
```

```
Future<void> _initializeTextToSpeech() async {
 flutterTts = FlutterTts();
```

```
 await flutterTts.setLanguage('en-US');
 await flutterTts.setSpeechRate(0.8);
 await flutterTts.setVolume(1.0);
 await flutterTts.setPitch(1.0);
}
```

```
// Main voice listening loop
```

```
Future<String?> listenForCommand({
 int timeoutSeconds = 10,
}) async {
 try {
 isListening.value = true;
 voiceInput.value = ";
```

```
 await speechToText.listen(
 onResult: (result) {
 voiceInput.value = result.recognizedWords;
 if (result.finalResult) {
 isListening.value = false;
 }
 },
 listenMode: ListenMode.search,
```

```

 cancelOnError: true,
 partialResults: true,
 onSoundLevelChange: (level) {
 // Could visualize sound level
 },
);

 // Wait for timeout or result
 await Future.delayed(Duration(seconds: timeoutSeconds));
 await speechToText.stop();

 return voiceInput.value.isEmpty ? null : voiceInput.value;
 } catch (e) {
 voiceError.value = 'Error listening: $e';
 return null;
 }
 }

 // Text-to-speech response
 Future<void> speak(String text, {
 bool wait = false,
 }) async {
 try {
 await flutterTts.speak(text);

 if (wait) {
 // Wait for speech to complete
 await Future.delayed(
 Duration(milliseconds: (text.length * 100).toInt())
);
 }
 } catch (e) {
 voiceError.value = 'Error speaking: $e';
 }
 }

 // Haptic feedback on voice trigger
 Future<void> hapticFeedback() async {
 await HapticFeedback.mediumImpact();
 }

 void dispose() {
 speechToText.stop();
 flutterTts.stop();
 super.dispose();
 }
}

```

```

// Voice Command Parser - converts speech to intents
class VoiceCommandParser {
 final nlpService = Get.find<NLPService>();

 Future<VoiceIntent?> parseCommand(String voiceInput) async {
 if (voiceInput.isEmpty) return null;

 // Send to NLP (user's configured LLM API)
 final response = await nlpService.parseIntent(
 input: voiceInput,
 context: getCurrentJobContext(),
);

 if (response == null) return null;

 return VoiceIntent.fromJson(response);
 }

 Map<String, dynamic> getCurrentJobContext() {
 // Provide context about current job to help NLP
 final currentJob = Get.find<JobController>().currentJob.value;

 if (currentJob == null) {
 return {'context': 'no_active_job'};
 }

 return {
 'jobId': currentJob.id,
 'clientId': currentJob.clientId,
 'serviceType': currentJob.serviceType,
 'status': currentJob.status.name,
 'location': currentJob.serviceLocation.address,
 };
 }
}

class VoiceIntent {
 final String action; // 'accept_job', 'navigate', 'complete', etc
 final Map<String, dynamic> parameters;
 final double confidence; // 0-1

 VoiceIntent({
 required this.action,
 required this.parameters,
 required this.confidence,
 });

 factory VoiceIntent.fromJson(Map<String, dynamic> json) {

```

```

return VoiceIntent(
 action: json['action'],
 parameters: json['parameters'] ?? {},
 confidence: (json['confidence'] as num).toDouble(),
);
}
}

```

```

// Voice Command Handler - executes voice intents
class VoiceCommandHandler extends GetxService {
 final voiceEngine = Get.find<VoiceEngine>();
 final commandParser = Get.find<VoiceCommandParser>();
 final titanZero = Get.find<TitanZeroRequestRouter>();
 final jobController = Get.find<JobController>();
 final navigationController = Get.find<NavigationController>();

```

```

Future<void> handleVoiceCommand() async {
 // 1. Listen for input
 voiceEngine.hapticFeedback();
 final voiceInput = await voiceEngine.listenForCommand();

 if (voiceInput == null || voiceInput.isEmpty) {
 voiceEngine.speak('Sorry, I didn\'t hear that. Please try again.');
```

```

 return;
 }

 // 2. Parse to intent
 final intent = await commandParser.parseCommand(voiceInput);

 if (intent == null || intent.confidence < 0.6) {
 voiceEngine.speak('I didn\'t understand that. Could you repeat?');
 return;
 }

 // 3. Execute via Titan Zero
 await executeIntent(intent);
}

```

```

Future<void> executeIntent(VoiceIntent intent) async {
 try {
 switch (intent.action) {
 // Job Management
 case 'accept_job':
 await jobController.acceptCurrentJob();
 voiceEngine.speak('Job accepted. Directions loaded.');
```

```

 break;

 case 'decline_job':

```

```
 await jobController.declineCurrentJob();
 voiceEngine.speak('Job declined.');
```

break;

```
case 'arrive_location':
 await jobController.updateStatus(JobStatus.arrived);
 voiceEngine.speak('Marked as arrived. Check job details.');
```

break;

```
case 'start_work':
 await jobController.updateStatus(JobStatus.in_progress);
 voiceEngine.speak('Work started. Timer running.');
```

break;

```
case 'complete_job':
 voiceEngine.speak('Please provide job completion details.');
```

await navigateToCompletion();

break;

```
// Navigation
case 'navigate':
 await navigationController.startNavigation();
 voiceEngine.speak('Navigation started. Follow the route.');
```

break;

```
case 'show_directions':
 navigationController.toggleDirections();
 voiceEngine.speak('Directions displayed.');
```

break;

```
// Information
case 'job_details':
 voiceEngine.speak(getJobDetailsSummary());
```

break;

```
case 'customer_info':
 voiceEngine.speak(getCustomerInfoSummary());
```

break;

```
case 'time_status':
 voiceEngine.speak(getTimeStatus());
```

break;

```
// Communication
case 'call_office':
 await makeCall('office');
```

break;

```

 case 'call_customer':
 await makeCall('customer');
 break;

 case 'send_message':
 await sendMessage(intent.parameters['message'] as String);
 break;

 // Photos
 case 'take_photo':
 voiceEngine.speak('Opening camera for photo.');
 await captureProofPhoto();
 break;

 case 'review_photos':
 voiceEngine.speak('${getPhotoCount()} photos taken.');
```

```

 break;

 default:
 voiceEngine.speak('Command not recognized: ${intent.action}');
 }
} catch (e) {
 voiceEngine.speak('Error: ${e.toString()}');
}
}

```

```

String getJobDetailsSummary() {
 final job = jobController.currentJob.value;
 if (job == null) return 'No active job.';

 return ""
 Job: ${job.serviceType}
 Location: ${job.serviceLocation.address}
 Estimated time: ${job.estimatedDuration} hours
 Price: \${job.quotedPrice}
 "",
}

```

```

String getCustomerInfoSummary() {
 final job = jobController.currentJob.value;
 if (job == null) return 'No active job.';

 // Get customer name from clientId
 return 'Customer: ABC Corp. Contact: 555-1234';
}

```

```

String getTimeStatus() {
 final job = jobController.currentJob.value;

```

```

 if (job == null) return 'No active job.';

 final elapsed = DateTime.now().difference(job.scheduledTime);
 return 'Work time: ${elapsed.inMinutes} minutes.';
 }
}
...

```

### ### 0.3 Offline-First Architecture Setup (25 hours)

```

`dart
// Offline-first database with sync
class OfflineFirstDatabase extends GetxService {
 late Database db;
 late Hive hiveBox;

 @override
 Future<void> onInit() async {
 super.onInit();
 await initializeLocalDatabase();
 setupSyncEngine();
 }

 Future<void> initializeLocalDatabase() async {
 // SQLite for structured data
 final path = await getDatabasesPath();
 db = await openDatabase(
 join(path, 'titan_go.db'),
 version: 1,
 onCreate: _createTables,
);

 // Hive for fast cache
 hiveBox = await Hive.openBox('titan_go_cache');
 }

 void _createTables(Database db, int version) async {
 // Jobs table
 await db.execute("""
 CREATE TABLE jobs (
 id TEXT PRIMARY KEY,
 client_id TEXT NOT NULL,
 service_type TEXT NOT NULL,
 location_latitude REAL NOT NULL,
 location_longitude REAL NOT NULL,
 location_address TEXT NOT NULL,
 scheduled_time TEXT NOT NULL,
 status TEXT NOT NULL,

```



```
estimated_duration REAL NOT NULL,
quoted_price REAL NOT NULL,
completed_time TEXT,
completion_notes TEXT,
proof_images TEXT,
created_at TEXT NOT NULL,
synced INTEGER DEFAULT 0
);
"";
```

```
// Location tracking
await db.execute("
CREATE TABLE location_tracking (
 id TEXT PRIMARY KEY,
 job_id TEXT NOT NULL,
 latitude REAL NOT NULL,
 longitude REAL NOT NULL,
 accuracy REAL,
 timestamp TEXT NOT NULL,
 synced INTEGER DEFAULT 0
);
"";
```

```
// Work time entries
await db.execute("
CREATE TABLE time_entries (
 id TEXT PRIMARY KEY,
 job_id TEXT NOT NULL,
 start_time TEXT NOT NULL,
 end_time TEXT,
 duration_hours REAL,
 synced INTEGER DEFAULT 0
);
"";
```

```
// Photos/proof
await db.execute("
CREATE TABLE attachments (
 id TEXT PRIMARY KEY,
 job_id TEXT NOT NULL,
 file_path TEXT NOT NULL,
 file_type TEXT,
 uploaded INTEGER DEFAULT 0,
 synced INTEGER DEFAULT 0
);
"";
```

```
// Create indices
```

```

await db.execute('CREATE INDEX idx_jobs_status ON jobs(status)');
await db.execute('CREATE INDEX idx_jobs_scheduled ON jobs(scheduled_time)');
await db.execute('CREATE INDEX idx_tracking_job ON location_tracking(job_id)');
}

```

// Save data locally

```

Future<void> saveJob(Job job) async {
 await db.insert(
 'jobs',
 {
 'id': job.id,
 'client_id': job.clientId,
 'service_type': job.serviceType,
 'location_latitude': job.serviceLocation.latitude,
 'location_longitude': job.serviceLocation.longitude,
 'location_address': job.serviceLocation.address,
 'scheduled_time': job.scheduledTime.toIso8601String(),
 'status': job.status.name,
 'estimated_duration': job.estimatedDuration,
 'quoted_price': job.quotedPrice,
 'created_at': DateTime.now().toIso8601String(),
 'synced': 0, // Not synced yet
 },
 conflictAlgorithm: ConflictAlgorithm.replace,
);
}

```

// Sync when online

```

Future<void> syncWithServer() async {
 final networkInfo = Get.find<NetworkInfo>();

 if (!await networkInfo.isConnected()) {
 return; // Not online, will sync later
 }
}

```

// Get unsynced records

```

final unsyncedJobs = await db.query(
 'jobs',
 where: 'synced = ?',
 whereArgs: [0],
);

```

for (final jobData in unsyncedJobs) {

```

 try {
 // Send to server
 await apiClient.syncJob(jobData);

```

```

 // Mark as synced

```

```

 await db.update(
 'jobs',
 {'synced': 1},
 where: 'id = ?',
 whereArgs: [jobData['id']],
);
 } catch (e) {
 print('Sync error: $e'); // Retry later
 }
}
}

```

```

void setupSyncEngine() {
 // Monitor network connectivity
 final connectivity = Connectivity();
 connectivity.onConnectivityChanged.listen((result) {
 if (result != ConnectivityResult.none) {
 syncWithServer(); // Sync when connection restored
 }
 });

 // Periodic sync (every 5 minutes if online)
 Timer.periodic(Duration(minutes: 5), (_) {
 syncWithServer();
 });
}
}

```

```

// Network Info - knows if online/offline
class NetworkInfo extends GetxService {
 final isOnline = true.obs;

```

```

 @override
 void onInit() {
 super.onInit();
 _setupConnectivityListener();
 }

```

```

 void _setupConnectivityListener() {
 final connectivity = Connectivity();
 connectivity.onConnectivityChanged.listen((result) {
 isOnline.value = result != ConnectivityResult.none;

 if (isOnline.value) {
 // Trigger sync when online
 Get.find<OfflineFirstDatabase>().syncWithServer();
 }
 });
 }

```

```

}

Future<bool> isConnected() => Connectivity().checkConnectivity()
 .then((result) => result != ConnectivityResult.none);
}
...

```

**\*\*Foundation Checklist:\*\***

- [ ] API keys moved to environment variables
- [ ] Tokens encrypted with flutter\_secure\_storage
- [ ] Certificate pinning implemented
- [ ] Voice engine initialized & tested
- [ ] Voice command parser working
- [ ] Offline-first database created
- [ ] Sync engine operational
- [ ] All tests passing

---

**## PHASE 1: JOB DISPATCH & ACCEPTANCE (Weeks 4-6)**

**### Accept Jobs, View Details, Navigation**

**### 1.1 Job Dispatch Controller**

```

```dart
class JobDispatchController extends GetxController {
  final jobRepository = Get.find<JobRepository>();
  final voiceEngine = Get.find<VoiceEngine>();
  final locationService = Get.find<LocationService>();
  final titanZero = Get.find<TitanZeroRequestRouter>();

  final incomingJobs = <Job>[].obs;
  final acceptedJob = Rxn<Job>();
  final jobStatus = JobStatus.pending.obs;
  final isLoading = false.obs;

  @override
  void onInit() {
    super.onInit();
    loadIncomingJobs();
    setupRealTimeUpdates();
    setupVoiceListener();
  }

  Future<void> loadIncomingJobs() async {
    isLoading.value = true;

    // Load from local database first

```

```

final localJobs = await jobRepository.getUnassignedJobs();
incomingJobs.value = localJobs;

// Sync with server if online
if (await networkInfo.isConnected()) {
    final serverJobs = await jobRepository.fetchNewJobs();
    incomingJobs.value = serverJobs;
}

isLoading.value = false;

// Notify if new jobs
if (incomingJobs.isNotEmpty) {
    voiceEngine.speak('${incomingJobs.length} new jobs available');
}
}

void setupRealTimeUpdates() {
    // Listen for real-time job updates from server
    signalCore.on<NewJobAssignedEvent>((event) {
        final job = Job.fromJson(event.jobData);
        incomingJobs.add(job);

        // Notify serviceman
        voiceEngine.speak('New job: ${job.serviceType} at ${job.serviceLocation.address}');
    });
}

void setupVoiceListener() {
    // Listen for voice commands about jobs
    voiceEngine.onCommandDetected.listen((command) {
        if (command.contains('accept')) {
            acceptCurrentJob();
        } else if (command.contains('next')) {
            nextJob();
        } else if (command.contains('decline')) {
            declineCurrentJob();
        }
    });
}

Future<void> acceptCurrentJob() async {
    if (acceptedJob.value == null && incomingJobs.isNotEmpty) {
        acceptedJob.value = incomingJobs.first;
    }

    final job = acceptedJob.value;
    if (job == null) {

```

```

        voiceEngine.speak('No job selected');
        return;
    }

    isLoading.value = true;

    try {
        // Accept via LogiCore (validates acceptance is allowed)
        final validation = await titanZero.validateJobAcceptance(job.id);

        if (!validation.isValid) {
            voiceEngine.speak('Cannot accept: ${validation.reason}');
            return;
        }

        // Update status
        jobStatus.value = JobStatus.assigned;

        // Sync with server
        await jobRepository.acceptJob(job.id);

        // Remove from incoming list
        incomingJobs.remove(job);

        // Notify
        voiceEngine.speak('Job accepted. Navigate to ${job.serviceLocation.address}');

        // Load next job if available
        if (incomingJobs.isNotEmpty) {
            nextJob();
        }

        // Signal event
        signalCore.emit(JobAcceptedEvent(job.id));
    } catch (e) {
        voiceEngine.speak('Error accepting job: ${e.toString()}');
    }

    isLoading.value = false;
}

void nextJob() {
    if (incomingJobs.isEmpty) {
        voiceEngine.speak('No more jobs');
        return;
    }

    acceptedJob.value = incomingJobs.first;

```

```

    voiceEngine.speak('Job: ${acceptedJob.value?.serviceType}');
  }

Future<void> declineCurrentJob() async {
  final job = acceptedJob.value;
  if (job == null) return;

  try {
    await jobRepository.declineJob(job.id);
    incomingJobs.remove(job);
    acceptedJob.value = null;

    voiceEngine.speak('Job declined');
    nextJob();
  } catch (e) {
    voiceEngine.speak('Error declining job');
  }
}

```

```

class JobStatus {
  static const String pending = 'pending';
  static const String assigned = 'assigned';
  static const String en_route = 'en_route';
  static const String arrived = 'arrived';
  static const String in_progress = 'in_progress';
  static const String completed = 'completed';
  static const String cancelled = 'cancelled';
}
...

```

1.2 Job Details Screen

```

``dart
class JobDetailsScreen extends StatelessWidget {
  final controller = Get.put(JobDispatchController());

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: Text('Job Details'),
        actions: [
          IconButton(
            icon: Icon(Icons.mic),
            onPressed: () => Get.find<VoiceEngine>().listenForCommand(),
            tooltip: 'Voice Command',
          ),

```

```
    ],
  ),
  body: Obx(() {
    final job = controller.acceptedJob.value;

    if (job == null) {
      return Center(child: Text('No job selected'));
    }

    return SingleChildScrollView(
      padding: EdgeInsets.all(16),
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          // Service Type
          Card(
```